

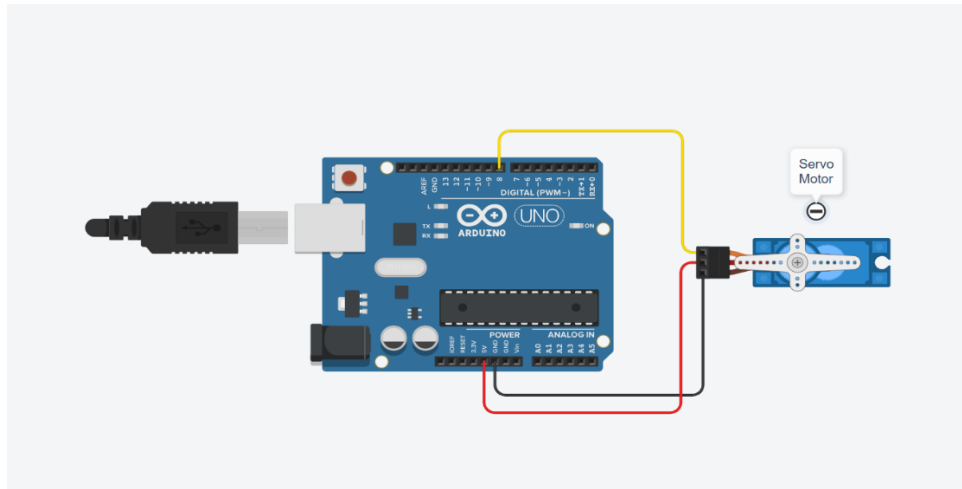


Figure 1: Servo Motor.

```
// --- Servo Motor Code ---  
#include <Servo.h>  
  
Servo myservo;  
  
void setup() {  
  myservo.attach(9); // Attach servo signal pin to digital pin 9  
}  
  
void loop() {  
  for (int pos = 0; pos <= 180; pos += 10) {  
    myservo.write(pos);  
    delay(500);  
  }  
  
  for (int pos = 180; pos >= 0; pos -= 10) {  
    myservo.write(pos);  
    delay(500);  
  }  
}
```

Expt No:

02

Date:

/ / 2025

ACTUATOR CONTROL AND PERFORMANCE ANALYSIS

1. AIM:

To control the angular position of a servo motor using Arduino and analyze its performance in terms of accuracy, response time, and stability and rotation of a stepper motor using Arduino and analyze its performance based on step angle, torque, and speed.

2. APPARATUS REQUIRED:

- a) Microcontroller: Arduino UNO R3,
- b) Actuator:
 - i) Servo Motor (SG90 or MG995),
 - ii) Stepper Motor (28BYJ-48 or NEMA 17).
- c) Software: Arduino IDE
- d) Components: Jumper Wires, USB-B Cable, Breadboard (optional),
- e) Driver Module: ULN2003 or A4988.

3. THEORY:

- a) Servo Motor: This is a closed-loop actuator that provides precise control of angular position. It receives a control signal (PWM) and adjusts its output shaft position accordingly using internal feedback from a potentiometer. The width of the input pulse determines the shaft angle. Pulse width range: Typically, 1ms (0°) to 2ms (180°), Feedback mechanism: Potentiometer inside the motor, Control signal: Generated by Arduino using PWM
- b) Stepper Motor: This is an electromechanical device that converts electrical pulses into discrete mechanical movements (steps). Each pulse moves the shaft by a fixed step angle, enabling open-loop position control. Stepper motors are commonly used in CNC machines, 3D printers, and precise positioning systems.

4. PROCEDURE:

- a) Servo Motor Control:
 - i) Connections:
 - 1) Servo Motor VCC → Arduino 5V,
 - 2) Servo Motor GND → Arduino GND,
 - 3) Servo Motor Signal → Arduino Digital Pin 9.
 - ii) Code: Upload the following sketch to the Arduino.
 - iii) Observation: Observe the servo motor shaft rotating from 0° to 180° and back to 0° in smooth steps. The motion should be precise and repeatable according to the PWM signal generated by the Arduino.

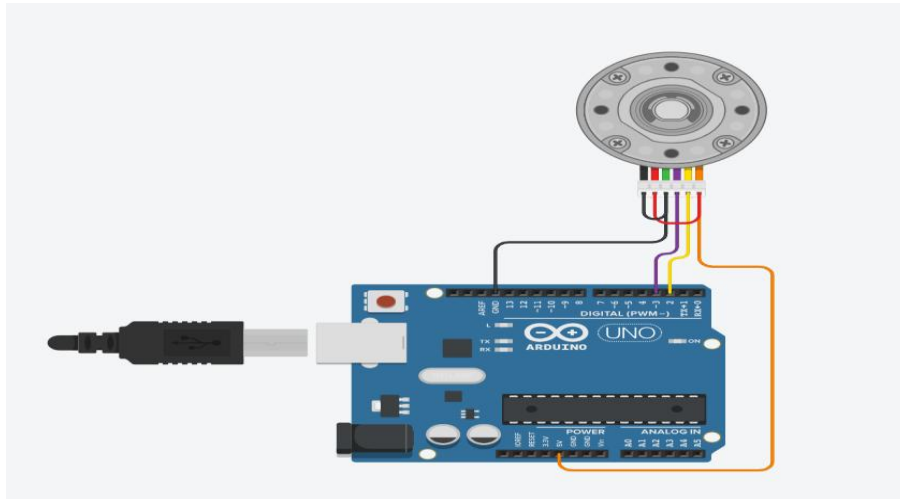


Figure 2: Stepper Motor.

```
// --- Stepper Motor Code ---
#include <Stepper.h>

#define IN1 8
#define IN2 9
#define IN3 10
#define IN4 11

#define stepsPerRevolution 2048 // This value may vary depending on your stepper motor

Stepper myStepper(stepsPerRevolution, IN1, IN2, IN3, IN4);

void setup() {
  myStepper.setSpeed(15);
  Serial.begin(9600);
  Serial.println("Stepper Motor Control");
}

void loop() {
  Serial.println("Clockwise");
  myStepper.step(stepsPerRevolution);
  delay(1000);

  Serial.println("Counterclockwise");
  myStepper.step(-stepsPerRevolution);
  delay(1000);
}
```

b) Stepper Motor Control:

i) Connections:

- 1) IN1 → Arduino Digital Pin 8,
- 2) IN2 → Arduino Digital Pin 9,
- 3) IN3 → Arduino Digital Pin 10,
- 4) IN4 → Arduino Digital Pin 11,
- 5) Driver Module VCC → Arduino 5V,
- 6) Driver Module GND → Arduino GND.

ii) Code: Upload the following sketch to the Arduino.

iii) Observation: Observe the stepper motor rotating first in the clockwise direction and then in the counterclockwise direction, with smooth stepwise motion corresponding to the programmed number of steps per revolution.

5. RESULT AND OBSERVATION:

Both the servo motor and the stepper motor were successfully controlled using the Arduino UNO. The experiments provided a clear, practical understanding of how to manage a servo motor's angular position, noting its accuracy, response time, and stability. Similarly, the control of the stepper motor demonstrated its precise, step-based rotation and the clear relationship between its speed, holding torque, and step angle. Both actuators responded reliably to the programmed commands, confirming the effectiveness of the control methods.